

ALGORITMA DAN STRUKTUR DATA
LATIHAN PRAKTIKUM 4 SINGEL LINKED LIST



Dosen Pembimbing:

Prof. Dr Arna Fariza S.Kom., M.Kom

Disusun oleh:

Nama: Luthfi Bahrur Rozaq

NRP: 3125600069

1 D4 TEKNIK INFORMATIKA C
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA

TUGAS PRAKTIKUM ASD – SINGLE LINKED LIST

1. Buatlah single linked list dengan data bertipe integer yang dapat melakukan operasi sisip secara terurut dan hapus simpul tertentu dengan ketentuan sebagai berikut :
 - a. Operasi sisip, buat sebuah simpul baru,
 - Jika data simpul baru < data pada head, maka gunakan sisip awal list
 - Jika pencarian data simpul baru mencapai NULL (data belum ada), sisip akhir list
 - Jika data simpul baru = data pada simpul tertentu maka berikan pesan simpul sudah ada (duplikat)
 - Lainnya sisip sebelum simpul tertentu
 - b. Operasi hapus,
 - Jika posisi simpul yang dihapus pada head, gunakan hapus awal list
 - Jika posisi simpul yang dihapus pada tail, gunakan hapus akhir list
 - Lainnya hapus simpul tengah

```
1. #include <stdio.h>
2. #include <stdlib.h>
3.
4. typedef struct simpul node;
5. struct simpul {
6.     int data;
7.     node *next;
8. };
9.
10. node *head = NULL;
11.
12. node *buat_simpul(int);
13. void free_node(node *);
14. void cetak_list();
15. void sisip_awal(int);
16. void sisip_akhir(int);
17. void sisip_sebelum(int, int);
18. void hapus_awal();
19. void hapus_akhir();
20. void hapus_tengah(int);
21.
22. int main() {
23.     char pilihan;
24.     int nilai;
25.
26.     do {
27.         printf("\noperasi singel linked list\n");
28.         printf("a. operasi sisip terurut\n");
```

```

29.     printf("b. operasi hapus\n");
30.     printf("c. tampilkan list\n");
31.     printf("x. keluar\n");
32.     printf("pilih menu: ");
33.     scanf(" %c", &pilihan);
34.
35.     switch (pilihan) {
36.         case 'A':
37.         case 'a': {
38.             printf("masukkan nilai yang disisipkan: ");
39.             scanf("%d", &nilai);
40.
41.             if (head == NULL || nilai < head->data) {
42.                 sisip_awal(nilai);
43.                 printf("menggunakan sisip awal\n");
44.             }
45.             else if (head->data == nilai) {
46.                 printf("pesan: simpul sudah ada
(duplikat)\n");
47.             }
48.             else {
49.                 node *before = head;
50.                 while (before->next != NULL && before-
>next->data < nilai) {
51.                     before = before->next;
52.                 }
53.
54.                 if (before->next == NULL) {
55.                     sisip_akhir(nilai);
56.                     printf("menggunakan sisip akhir.\n");
57.                 }
58.                 else if (before->next->data == nilai) {
59.                     printf("pesan: simpul sudah ada
(duplikat)\n");
60.                 }
61.                 else {
62.                     sisip_sebelum(nilai, before->next-
>data);
63.                     printf("menggunakan sisip sebelum
simpul tertentu\n");
64.                 }
65.             }
66.             break;

```

```

67.         }
68.
69.         case 'B':
70.         case 'b': {
71.             printf("masukkan nilai yang dihapus: ");
72.             scanf("%d", &nilai);
73.
74.             if (head == NULL) {
75.                 printf("list kosong!\n");
76.             }
77.             else if (head->data == nilai) {
78.                 hapus_awal();
79.                 printf("menggunakan hapus awal list\n");
80.             }
81.             else {
82.                 node *before = head;
83.                 while (before->next != NULL && before->
>next->data != nilai) {
84.                     before = before->next;
85.                 }
86.
87.                 if (before->next == NULL) {
88.                     printf("data tidak ditemukan!\n");
89.                 }
90.                 else if (before->next->next == NULL) {
91.                     hapus_akhir();
92.                     printf("menggunakan hapus akhir
list\n");
93.                 }
94.                 else {
95.                     hapus_tengah(nilai);
96.                     printf("menggunakan hapus simpul
tengah\n");
97.                 }
98.             }
99.             break;
100.        }
101.
102.        case 'C':
103.        case 'c': {
104.            cetak_list();
105.            break;
106.        }

```

```

107.
108.         case 'X':
109.         case 'x':
110.             printf("keluar dari program\n");
111.             break;
112.
113.         default:
114.             printf("pilihan tidak valid!\n");
115.     }
116. } while (pilihan != 'x' && pilihan != 'X');
117.
118. return 0;
119. }
120.
121. node *buat_simpul(int x) {
122.     node *baru = (node *) malloc(sizeof(node));
123.     baru->data = x;
124.     baru->next = NULL;
125.     return baru;
126. }
127.
128. void free_node(node *p) {
129.     free(p);
130.     p = NULL;
131. }
132.
133. void cetak_list() {
134.     node *p = head;
135.     printf("isi list: ");
136.     while (p != NULL) {
137.         printf("%d ", p->data);
138.         p = p->next;
139.     }
140.     printf("\n");
141. }
142.
143. void sisip_awal(int x) {
144.     node *baru = buat_simpul(x);
145.     baru->next = head;
146.     head = baru;
147. }
148.
149. void sisip_akhir(int x) {

```

```

150.     node *baru = buat_simpul(x);
151.     if (head == NULL) {
152.         head = baru;
153.         return;
154.     }
155.
156.     node *tail = head;
157.     while (tail->next != NULL) {
158.         tail = tail->next;
159.     }
160.     tail->next = baru;
161. }
162.
163. void sisip_sebelum(int x, int target) {
164.     node *baru = buat_simpul(x);
165.     node *before = head;
166.     while (before->next != NULL && before->next->data !=
target) {
167.         before = before->next;
168.     }
169.     baru->next = before->next;
170.     before->next = baru;
171. }
172.
173. void hapus_awal() {
174.     if (head == NULL) return;
175.     node *hapus = head;
176.     head = hapus->next;
177.     free_node(hapus);
178. }
179.
180. void hapus_akhir() {
181.     if (head == NULL) return;
182.     if (head->next == NULL) {
183.         hapus_awal();
184.         return;
185.     }
186.
187.     node *hapus = head, *before = NULL;
188.     while (hapus->next != NULL) {
189.         before = hapus;
190.         hapus = hapus->next;
191.     }

```

```

192.         before->next = NULL;
193.         free_node(hapus);
194.     }
195.
196.     void hapus_tengah(int x) {
197.         node *hapus = head, *before = NULL;
198.         while (hapus != NULL && hapus->data != x) {
199.             before = hapus;
200.             hapus = hapus->next;
201.         }
202.         if (hapus != NULL) {
203.             before->next = hapus->next;
204.             free_node(hapus);
205.         }
206.     }

```

Output:

```

operasi singel linked list
a. operasi sisip terurut
b. operasi hapus
c. tampilkan list
x. keluar
pilih menu: a
masukkan nilai yang disisipkan: 30
menggunakan sisip awal

```

```

operasi singel linked list
a. operasi sisip terurut
b. operasi hapus
c. tampilkan list
x. keluar
pilih menu: a
masukkan nilai yang disisipkan: 10
menggunakan sisip awal

```

```

operasi singel linked list
a. operasi sisip terurut
b. operasi hapus
c. tampilkan list
x. keluar
pilih menu: a
masukkan nilai yang disisipkan: 50
menggunakan sisip akhir.

```

```

operasi singel linked list
a. operasi sisip terurut
b. operasi hapus
c. tampilkan list
x. keluar
pilih menu: a
masukkan nilai yang disisipkan: 20
menggunakan sisip sebelum simpul tertentu

```

```

operasi singel linked list
a. operasi sisip terurut
b. operasi hapus
c. tampilkan list
x. keluar
pilih menu: a
masukkan nilai yang disisipkan: 30
pesan: simpul sudah ada (duplikat)

```

```

operasi singel linked list
a. operasi sisip terurut
b. operasi hapus
c. tampilkan list
x. keluar
pilih menu: c
isi list: 10 20 30 50

```

```

operasi singel linked list
a. operasi sisip terurut
b. operasi hapus
c. tampilkan list
x. keluar
pilih menu: b
masukkan nilai yang dihapus: 10
menggunakan hapus awal list

```

```

operasi singel linked list
a. operasi sisip terurut
b. operasi hapus
c. tampilkan list
x. keluar
pilih menu: x
keluar dari program

```

```

operasi singel linked list
a. operasi sisip terurut
b. operasi hapus
c. tampilkan list
x. keluar
pilih menu: b
masukkan nilai yang dihapus: 50
menggunakan hapus akhir list

operasi singel linked list
a. operasi sisip terurut
b. operasi hapus
c. tampilkan list
x. keluar
pilih menu: b
masukkan nilai yang dihapus: 100
data tidak ditemukan!

operasi singel linked list
a. operasi sisip terurut
b. operasi hapus
c. tampilkan list
x. keluar
x. keluar
pilih menu: c
isi list: 20 30

```

Analisis Program:

Program ini mendemonstrasikan implementasi dasar memori dinamis menggunakan Single Linked List dengan alur yang dimulai dari proses penyisipan data secara terurut, di mana algoritma melakukan penelusuran menggunakan pointer bantuan untuk membandingkan nilai dan berhenti tepat satu langkah sebelum titik penyisipan agar rantai list tidak terputus saat menyambungkan data baru, sekaligus menangani berbagai skenario khusus seperti list kosong, penyisipan di awal, penyisipan di akhir, hingga pemblokiran input duplikat, yang kemudian dilanjutkan dengan operasi penghapusan data di mana program secara berurutan mencari lokasi target terlebih dahulu untuk kemudian menyambungkan pointer dari node sebelum target langsung ke node sesudahnya sehingga node target terlepas dari rantai dan dapat dikembalikan ke sistem memori menggunakan perintah free.

2. Merepresentasikan sebuah bilangan polinomial dengan single linked list. Masalah aritmatika polinom adalah membuat sekumpulan subrutin manipulasi terhadap polinom simbolis (symbolic Polynomial).

Misalnya: $P1 = 6x^8 + 8x^7 + 5x^5 + x^3 + 15$
 $P2 = 3x^9 + 4x^7 + 3x^4 + 2x^3 + 2x^2 + 10$

Representasikan bilangan polinom dengan menggunakan linked list dan buatlah prosedurprosedur untuk :

- Menyisipkan simpul di awal jika pangkat yang dimasukkan lebih dari pangkat tertinggi dari bilangan polinomial.
- Menyisipkan simpul di tengah jika pangkat dari bilangan yang kita sisipkan berada di tengah.
- Menyisipkan simpul di akhir jika pangkat dari bilangan yang disisipkan adalah 0.

- Menghapus simpul, baik di awal, di tengah, ataupun di akhir.

```
1. #include <stdio.h>
2. #include <stdlib.h>
3.
4. typedef struct simpul node;
5. struct simpul {
6.     int koef;
7.     int pangkat;
8.     node *next;
9. };
10.
11. node *head = NULL;
12.
13. node *buat_simpul(int, int);
14. void free_node(node *);
15. void cetak_list();
16. void sisip_awal(int, int);
17. void sisip_akhir(int, int);
18. void sisip_sebelum(int, int, int);
19. void hapus_awal();
20. void hapus_akhir();
21. void hapus_tengah(int);
22.
23. int main() {
24.     char pilihan;
25.     int koef, pangkat;
26.
27.     do {
28.         printf("\noperasi polinomial\n");
29.         printf("a. operasi sisip terurut (pangkat)\n");
30.         printf("b. operasi hapus\n");
31.         printf("c. tampilkan polinomial\n");
32.         printf("x. keluar\n");
33.         printf("pilih menu: ");
34.         scanf(" %c", &pilihan);
35.
36.         switch (pilihan) {
37.             case 'A':
38.             case 'a': {
39.                 printf("masukkan koefisien: ");
40.                 scanf("%d", &koef);
41.                 printf("masukkan pangkat: ");
```

```

42.         scanf("%d", &pangkat);
43.
44.         if (head == NULL || pangkat > head->pangkat)
45.         {
46.             sisip_awal(koef, pangkat);
47.             printf("menggunakan sisip awal\n");
48.         }
49.         else if (head->pangkat == pangkat) {
50.             printf("pesan: pangkat sudah ada\n");
51.         }
52.         else {
53.             node *before = head;
54.             while (before->next != NULL && before->
55.                 >next->pangkat > pangkat) {
56.                 before = before->next;
57.             }
58.             if (before->next == NULL) {
59.                 sisip_akhir(koef, pangkat);
60.                 printf("menggunakan sisip akhir\n");
61.             }
62.             else if (before->next->pangkat ==
63.                 pangkat) {
64.                 printf("pesan: pangkat sudah ada\n");
65.             }
66.             else {
67.                 sisip_sebelum(koef, pangkat, before->
68.                 >next->pangkat);
69.                 printf("menggunakan sisip di
70.                 tengah\n");
71.             }
72.         }
73.         break;
74.     }
75.
76.     case 'B':
77.     case 'b': {
78.         printf("masukkan pangkat yang dihapus: ");
79.         scanf("%d", &pangkat);
80.
81.         if (head == NULL) {
82.             printf("list kosong!\n");
83.         }

```

```

80.         else if (head->pangkat == pangkat) {
81.             hapus_awal();
82.             printf("menggunakan hapus awal list\n");
83.         }
84.         else {
85.             node *before = head;
86.             while (before->next != NULL && before->next->pangkat != pangkat) {
87.                 before = before->next;
88.             }
89.
90.             if (before->next == NULL) {
91.                 printf("pangkat tidak ditemukan!\n");
92.             }
93.             else if (before->next->next == NULL) {
94.                 hapus_akhir();
95.                 printf("menggunakan hapus akhir
list\n");
96.             }
97.             else {
98.                 hapus_tengah(pangkat);
99.                 printf("menggunakan hapus simpul
tengah\n");
100.            }
101.        }
102.        break;
103.    }
104.
105.    case 'C':
106.    case 'c': {
107.        cetak_list();
108.        break;
109.    }
110.
111.    case 'X':
112.    case 'x':
113.        printf("keluar dari program\n");
114.        break;
115.
116.    default:
117.        printf("pilihan tidak valid!\n");
118.    }
119. } while (pilihan != 'x' && pilihan != 'X');

```

```

120.
121.     return 0;
122. }
123.
124. node *buat_simpul(int k, int p) {
125.     node *baru = (node *) malloc(sizeof(node));
126.     baru->koef = k;
127.     baru->pangkat = p;
128.     baru->next = NULL;
129.     return baru;
130. }
131.
132. void free_node(node *p) {
133.     free(p);
134.     p = NULL;
135. }
136.
137. void cetak_list() {
138.     node *p = head;
139.     printf("polinomial: ");
140.     while (p != NULL) {
141.         printf("%dx^%d ", p->koef, p->pangkat);
142.         if (p->next != NULL) printf("+ ");
143.         p = p->next;
144.     }
145.     printf("\n");
146. }
147.
148. void sisip_awal(int k, int p) {
149.     node *baru = buat_simpul(k, p);
150.     baru->next = head;
151.     head = baru;
152. }
153.
154. void sisip_akhir(int k, int p) {
155.     node *baru = buat_simpul(k, p);
156.     if (head == NULL) {
157.         head = baru;
158.         return;
159.     }
160.     node *tail = head;
161.     while (tail->next != NULL) {
162.         tail = tail->next;

```

```

163.     }
164.     tail->next = baru;
165. }
166.
167. void sisip_sebelum(int k, int p, int target) {
168.     node *baru = buat_simpul(k, p);
169.     node *before = head;
170.     while (before->next != NULL && before->next->pangkat
    != target) {
171.         before = before->next;
172.     }
173.     baru->next = before->next;
174.     before->next = baru;
175. }
176.
177. void hapus_awal() {
178.     if (head == NULL) return;
179.     node *hapus = head;
180.     head = hapus->next;
181.     free_node(hapus);
182. }
183.
184. void hapus_akhir() {
185.     if (head == NULL) return;
186.     if (head->next == NULL) {
187.         hapus_awal();
188.         return;
189.     }
190.     node *hapus = head, *before = NULL;
191.     while (hapus->next != NULL) {
192.         before = hapus;
193.         hapus = hapus->next;
194.     }
195.     before->next = NULL;
196.     free_node(hapus);
197. }
198.
199. void hapus_tengah(int target) {
200.     node *hapus = head, *before = NULL;
201.     while (hapus != NULL && hapus->pangkat != target) {
202.         before = hapus;
203.         hapus = hapus->next;
204.     }

```

```

205.         if (hapus != NULL) {
206.             before->next = hapus->next;
207.             free_node(hapus);
208.         }
209.     }

```

Output:

```

operasi polinomial
a. operasi sisip terurut (pangkat)
b. operasi hapus
c. tampilkan polinomial
x. keluar
pilih menu: a
masukkan koefisien: 5
masukkan pangkat: 3
menggunakan sisip awal

```

```

operasi polinomial
a. operasi sisip terurut (pangkat)
b. operasi hapus
c. tampilkan polinomial
x. keluar
pilih menu: a
masukkan koefisien: 2
masukkan pangkat: 5
menggunakan sisip awal

```

```

operasi polinomial
a. operasi sisip terurut (pangkat)
b. operasi hapus
c. tampilkan polinomial
x. keluar
pilih menu: a
masukkan koefisien: 10
masukkan pangkat: 0
menggunakan sisip akhir

```

```

operasi polinomial
a. operasi sisip terurut (pangkat)
b. operasi hapus
c. tampilkan polinomial
x. keluar
pilih menu: a
masukkan koefisien: 4
masukkan pangkat: 2
menggunakan sisip di tengah

```

```

operasi polinomial
a. operasi sisip terurut (pangkat)
b. operasi hapus
c. tampilkan polinomial
x. keluar
pilih menu: a
masukkan koefisien: 8
masukkan pangkat: 3
pesan: pangkat sudah ada

```

```

operasi polinomial
a. operasi sisip terurut (pangkat)
b. operasi hapus
c. tampilkan polinomial
x. keluar
pilih menu: c
polinomial:  $2x^5 + 5x^3 + 4x^2 + 10x^0$ 

```

```

operasi polinomial
x. keluar
pilih menu: b
masukkan pangkat yang dihapus: 5
menggunakan hapus awal list

```

```

operasi polinomial
a. operasi sisip terurut (pangkat)
b. operasi hapus
c. tampilkan polinomial
x. keluar
pilih menu: c
polinomial:  $5x^3 + 4x^2 + 10x^0$ 

```

```

operasi polinomial
a. operasi sisip terurut (pangkat)
b. operasi hapus
c. tampilkan polinomial
x. keluar
pilih menu: x
keluar dari program

```

Analisis Program:

Representasi polinomial ini mengadaptasi struktur Single Linked List untuk menyimpan persamaan matematika secara jauh lebih efisien memori dibandingkan array karena alokasi node hanya dilakukan untuk suku polinomial yang benar-benar memiliki koefisien, di mana alur kerjanya diawali dengan memodifikasi struktur node agar dapat menampung dua variabel pendamping sekaligus yaitu koefisien dan pangkat, lalu menerapkan siklus penyisipan yang meminjam logika program pertama namun dengan modifikasi parameter komparasi menjadi urutan menurun, sehingga setiap data baru yang masuk akan otomatis tersusun rapi dari pangkat tertinggi di posisi awal list menyusut hingga ke pangkat terendah atau nol di bagian ujung akhir list.

3. Implementasikan Stack dengan single linked list, buatlah program untuk konversi dari desimal ke biner, oktal, dan heksa.

```
1. #include <stdio.h>
2. #include <stdlib.h>
3.
4. typedef struct simpul node;
5. struct simpul {
6.     int data;
7.     node *next;
8. };
9.
10. node *head = NULL;
11.
12. node *buat_simpul(int);
13. void free_node(node *);
14. void push(int);
15. int pop();
16. void konversi(int, int);
17.
18. int main() {
19.     char pilihan;
20.     int desimal;
21.
22.     do {
23.         printf("\noperasi stack konversi basis\n");
24.         printf("a. desimal ke biner\n");
25.         printf("b. desimal ke oktal\n");
26.         printf("c. desimal ke heksadesimal\n");
27.         printf("x. keluar\n");
28.         printf("pilih menu: ");
```

```

29.     scanf(" %c", &pilihan);
30.
31.     if (pilihan == 'x' || pilihan == 'X') {
32.         printf("keluar dari program\n");
33.         break;
34.     }
35.
36.     if (pilihan == 'a' || pilihan == 'A' ||
37.         pilihan == 'b' || pilihan == 'B' ||
38.         pilihan == 'c' || pilihan == 'C') {
39.         printf("masukkan bilangan desimal: ");
40.         scanf("%d", &desimal);
41.     }
42.
43.     switch (pilihan) {
44.         case 'A':
45.         case 'a':
46.             konversi(desimal, 2);
47.             break;
48.         case 'B':
49.         case 'b':
50.             konversi(desimal, 8);
51.             break;
52.         case 'C':
53.         case 'c':
54.             konversi(desimal, 16);
55.             break;
56.         default:
57.             printf("pilihan tidak valid!\n");
58.     }
59. } while (1);
60.
61. return 0;
62.}
63.
64.node *buat_simpul(int x) {
65.     node *baru = (node *) malloc(sizeof(node));
66.     baru->data = x;
67.     baru->next = NULL;
68.     return baru;
69.}
70.
71.void free_node(node *p) {

```



```

72.     free(p);
73.     p = NULL;
74. }
75.
76. void push(int x) {
77.     node *baru = buat_simpul(x);
78.     baru->next = head;
79.     head = baru;
80. }
81.
82. int pop() {
83.     if (head == NULL) return -1;
84.     node *hapus = head;
85.     int nilai = hapus->data;
86.     head = hapus->next;
87.     free_node(hapus);
88.     return nilai;
89. }
90.
91. void konversi(int angka, int basis) {
92.     if (angka == 0) {
93.         push(0);
94.     } else {
95.         while (angka > 0) {
96.             push(angka % basis);
97.             angka = angka / basis;
98.         }
99.     }
100.
101.     printf("hasil: ");
102.     while (head != NULL) {
103.         int hasil = pop();
104.         if (hasil < 10) {
105.             printf("%d", hasil);
106.         } else {
107.             printf("%c", hasil + 55);
108.         }
109.     }
110.     printf("\n");
111. }

```

Output:

```
operasi stack konversi basis
a. desimal ke biner
b. desimal ke oktal
c. desimal ke heksadesimal
x. keluar
pilih menu: a
masukkan bilangan desimal: 13
hasil: 1101
```

```
operasi stack konversi basis
a. desimal ke biner
b. desimal ke oktal
c. desimal ke heksadesimal
x. keluar
pilih menu: b
masukkan bilangan desimal: 25
hasil: 31
```

```
operasi stack konversi basis
a. desimal ke biner
b. desimal ke oktal
c. desimal ke heksadesimal
x. keluar
pilih menu: c
masukkan bilangan desimal: 254
hasil: FE
```

```
operasi stack konversi basis
a. desimal ke biner
b. desimal ke oktal
c. desimal ke heksadesimal
x. keluar
pilih menu: a
masukkan bilangan desimal: 0
hasil: 0
```

```
operasi stack konversi basis
a. desimal ke biner
b. desimal ke oktal
c. desimal ke heksadesimal
x. keluar
pilih menu: x
keluar dari program
```

Analisis Program:

Program ini membuktikan fleksibilitas Single Linked List yang dapat dikonstruksi menjadi struktur Abstract Data Type berupa Stack berkonsep Last In First Out (LIFO) melalui pembatasan akses manipulasi data yang hanya diizinkan secara ketat pada satu titik akses saja, di mana alur program memanfaatkan fungsi sisip awal sebagai operasi Push untuk memasukkan data dan fungsi hapus awal sebagai operasi Pop untuk mengeluarkan data demi menyelesaikan perhitungan konversi bilangan desimal ke biner, oktal, maupun heksadesimal, karena karakteristik alami pemanggilan Stack secara otomatis mampu menyimpan sementara dan membalikkan urutan cetak sisa bagi kalkulasi modulus secara presisi tanpa memerlukan trik indeks perulangan mundur.

4. Buatlah single linked list dengan simpul berupa data mahasiswa yang terdiri dari NRP, Nama dan Kelas. Buatlah operasi Sisip secara terurut berdasarkan NRP, Hapus data mahasiswa tertentu berdasarkan NRP dan Update data berdasarkan NRP (nama dan kelas saja)

1. `#include <stdio.h>`
2. `#include <stdlib.h>`
3. `#include <string.h>`
- 4.
5. `typedef struct simpul node;`

```

6. struct simpul {
7.     int nrp;
8.     char nama[50];
9.     char kelas[20];
10.    node *next;
11.};
12.
13.node *head = NULL;
14.
15.node *buat_simpul(int, char*, char*);
16.void free_node(node *);
17.void cetak_list();
18.void sisip_awal(int, char*, char*);
19.void sisip_akhir(int, char*, char*);
20.void sisip_sebelum(int, char*, char*, int);
21.void hapus_awal();
22.void hapus_akhir();
23.void hapus_tengah(int);
24.void update_data(int, char*, char*);
25.
26.int main() {
27.    char pilihan;
28.    int nrp;
29.    char nama[50], kelas[20];
30.
31.    do {
32.        printf("\noperasi linked list mahasiswa\n");
33.        printf("a. operasi sisip terurut nrp\n");
34.        printf("b. operasi hapus\n");
35.        printf("c. update data\n");
36.        printf("d. tampilkan list\n");
37.        printf("x. keluar\n");
38.        printf("pilih menu: ");
39.        scanf(" %c", &pilihan);
40.
41.        switch (pilihan) {
42.            case 'A':
43.            case 'a': {
44.                printf("masukkan nrp: ");
45.                scanf("%d", &nrp);
46.                getchar();
47.                printf("masukkan nama: ");
48.                fgets(nama, 50, stdin);

```

```

49.         nama[strcspn(nama, "\n")] = 0;
50.         printf("masukkan kelas: ");
51.         fgets(kelas, 20, stdin);
52.         kelas[strcspn(kelas, "\n")] = 0;
53.
54.         if (head == NULL || nrp < head->nrp) {
55.             sisip_awal(nrp, nama, kelas);
56.             printf("menggunakan sisip awal\n");
57.         }
58.         else if (head->nrp == nrp) {
59.             printf("pesan: nrp sudah ada
(duplikat)\n");
60.         }
61.         else {
62.             node *before = head;
63.             while (before->next != NULL && before->
next->nrp < nrp) {
64.                 before = before->next;
65.             }
66.
67.             if (before->next == NULL) {
68.                 sisip_akhir(nrp, nama, kelas);
69.                 printf("menggunakan sisip akhir\n");
70.             }
71.             else if (before->next->nrp == nrp) {
72.                 printf("pesan: nrp sudah ada
(duplikat)\n");
73.             }
74.             else {
75.                 sisip_sebelum(nrp, nama, kelas,
before->next->nrp);
76.                 printf("menggunakan sisip sebelum nrp
tertentu\n");
77.             }
78.         }
79.         break;
80.     }
81.
82.     case 'B':
83.     case 'b': {
84.         printf("masukkan nrp yang dihapus: ");
85.         scanf("%d", &nrp);
86.

```

```

87.         if (head == NULL) {
88.             printf("list kosong!\n");
89.         }
90.         else if (head->nrp == nrp) {
91.             hapus_awal();
92.             printf("menggunakan hapus awal list\n");
93.         }
94.         else {
95.             node *before = head;
96.             while (before->next != NULL && before->
>next->nrp != nrp) {
97.                 before = before->next;
98.             }
99.
100.            if (before->next == NULL) {
101.                printf("data tidak
ditemukan!\n");
102.            }
103.            else if (before->next->next == NULL)
{
104.                hapus_akhir();
105.                printf("menggunakan hapus akhir
list\n");
106.            }
107.            else {
108.                hapus_tengah(nrp);
109.                printf("menggunakan hapus simpul
tengah\n");
110.            }
111.        }
112.        break;
113.    }
114.
115.    case 'C':
116.    case 'c': {
117.        printf("masukkan nrp yang ingin
diupdate: ");
118.        scanf("%d", &nrp);
119.        getchar();
120.        printf("masukkan nama baru: ");
121.        fgets(nama, 50, stdin);
122.        nama[strcspn(nama, "\n")] = 0;
123.        printf("masukkan kelas baru: ");

```

```

124.         fgets(kelas, 20, stdin);
125.         kelas[strcspn(kelas, "\n")] = 0;
126.
127.         update_data(nrp, nama, kelas);
128.         break;
129.     }
130.
131.     case 'D':
132.     case 'd': {
133.         cetak_list();
134.         break;
135.     }
136.
137.     case 'X':
138.     case 'x':
139.         printf("keluar dari program\n");
140.         break;
141.
142.     default:
143.         printf("pilihan tidak valid!\n");
144.     }
145. } while (pilihan != 'x' && pilihan != 'X');
146.
147. return 0;
148. }
149.
150. node *buat_simpul(int n, char* nm, char* kl) {
151.     node *baru = (node *) malloc(sizeof(node));
152.     baru->nrp = n;
153.     strcpy(baru->nama, nm);
154.     strcpy(baru->kelas, kl);
155.     baru->next = NULL;
156.     return baru;
157. }
158.
159. void free_node(node *p) {
160.     free(p);
161.     p = NULL;
162. }
163.
164. void cetak_list() {
165.     node *p = head;
166.     printf("\ndata mahasiswa\n");

```

```

167.         while (p != NULL) {
168.             printf("nrp: %d | nama: %s | kelas: %s\n", p-
>nrp, p->nama, p->kelas);
169.             p = p->next;
170.         }
171.         printf("\n");
172.     }
173.
174.     void sisip_awal(int n, char* nm, char* kl) {
175.         node *baru = buat_simpul(n, nm, kl);
176.         baru->next = head;
177.         head = baru;
178.     }
179.
180.     void sisip_akhir(int n, char* nm, char* kl) {
181.         node *baru = buat_simpul(n, nm, kl);
182.         if (head == NULL) {
183.             head = baru;
184.             return;
185.         }
186.         node *tail = head;
187.         while (tail->next != NULL) {
188.             tail = tail->next;
189.         }
190.         tail->next = baru;
191.     }
192.
193.     void sisip_sebelum(int n, char* nm, char* kl, int
target) {
194.         node *baru = buat_simpul(n, nm, kl);
195.         node *before = head;
196.         while (before->next != NULL && before->next->nrp !=
target) {
197.             before = before->next;
198.         }
199.         baru->next = before->next;
200.         before->next = baru;
201.     }
202.
203.     void hapus_awal() {
204.         if (head == NULL) return;
205.         node *hapus = head;
206.         head = hapus->next;

```

```

207.         free_node(hapus);
208.     }
209.
210.     void hapus_akhir() {
211.         if (head == NULL) return;
212.         if (head->next == NULL) {
213.             hapus_awal();
214.             return;
215.         }
216.         node *hapus = head, *before = NULL;
217.         while (hapus->next != NULL) {
218.             before = hapus;
219.             hapus = hapus->next;
220.         }
221.         before->next = NULL;
222.         free_node(hapus);
223.     }
224.
225.     void hapus_tengah(int target) {
226.         node *hapus = head, *before = NULL;
227.         while (hapus != NULL && hapus->nrp != target) {
228.             before = hapus;
229.             hapus = hapus->next;
230.         }
231.         if (hapus != NULL) {
232.             before->next = hapus->next;
233.             free_node(hapus);
234.         }
235.     }
236.
237.     void update_data(int n, char* nm, char* kl) {
238.         node *p = head;
239.         while (p != NULL && p->nrp != n) {
240.             p = p->next;
241.         }
242.         if (p != NULL) {
243.             strcpy(p->nama, nm);
244.             strcpy(p->kelas, kl);
245.             printf("data nrp %d berhasil diupdate.\n", n);
246.         } else {
247.             printf("nrp tidak ditemukan!\n");
248.         }
249.     }

```


Output:

```
operasi linked list mahasiswa
a. operasi sisip terurut nrp
b. operasi hapus
c. update data
d. tampilkan list
x. keluar
pilih menu: a
masukkan nrp: 200
masukkan nama: budi
masukkan kelas: 1 d4it a
menggunakan sisip awal
```

```
operasi linked list mahasiswa
a. operasi sisip terurut nrp
b. operasi hapus
c. update data
d. tampilkan list
x. keluar
pilih menu: a
masukkan nrp: 100
masukkan nama: andi
masukkan kelas: 1 d4it a
menggunakan sisip awal
```

```
operasi linked list mahasiswa
a. operasi sisip terurut nrp
b. operasi hapus
c. update data
d. tampilkan list
x. keluar
pilih menu: a
masukkan nrp: 300
masukkan nama: citra
masukkan kelas: 1 d4it b
menggunakan sisip akhir
```

```
operasi linked list mahasiswa
a. operasi sisip terurut nrp
b. operasi hapus
c. update data
d. tampilkan list
x. keluar
pilih menu: d

data mahasiswa
nrp: 100 | nama: andi | kelas: 1 d4it a
nrp: 200 | nama: budi | kelas: 1 d4it a
nrp: 300 | nama: citra | kelas: 1 d4it b
```

```
operasi linked list mahasiswa
a. operasi sisip terurut nrp
b. operasi hapus
c. update data
d. tampilkan list
x. keluar
pilih menu: c
masukkan nrp yang ingin diupdate: 200
masukkan nama baru: budi santoso
masukkan kelas baru: 2 d4it a
data nrp 200 berhasil diupdate.
```

```
operasi linked list mahasiswa
a. operasi sisip terurut nrp
b. operasi hapus
c. update data
d. tampilkan list
x. keluar
pilih menu: d

data mahasiswa
nrp: 100 | nama: andi | kelas: 1 d4it a
nrp: 200 | nama: budi santoso | kelas: 2 d4it a
nrp: 300 | nama: citra | kelas: 1 d4it b
```

```
operasi linked list mahasiswa
a. operasi sisip terurut nrp
b. operasi hapus
c. update data
d. tampilkan list
x. keluar
pilih menu: b
masukkan nrp yang dihapus: 100
menggunakan hapus awal list
```

```
operasi linked list mahasiswa
a. operasi sisip terurut nrp
b. operasi hapus
c. update data
d. tampilkan list
x. keluar
pilih menu: x
keluar dari program
```

Analisis Program:

Program terakhir ini membuat Single Linked List yang lebih kompleks untuk mengelola tipe data bentukan yang kompleks dengan cara menyatukan entitas rekaman mahasiswa yang mencakup NRP bertipe integer sebagai acuan parameter pengurutan beserta atribut tambahan berupa string nama dan kelas ke dalam satu node utuh, di mana alur programnya diawali dengan mengurutkan data masuk murni berdasarkan perbandingan angka NRP, dilanjutkan dengan kemampuan menyediakan fitur pembaruan (update) yang mendemonstrasikan bahwa manipulasi list tidak terbatas pada pembongkaran rute pointer saja, melainkan mampu melakukan penyimpanan konten teks menggunakan fungsi penyalinan string langsung pada alamat memori node spesifik yang dicari tanpa merusak susunan rantai memori di sekitarnya.

Kesimpulan

Pelaksanaan praktikum Algoritma dan Struktur Data mengenai Single Linked List (SLL) ini secara komprehensif membuktikan bahwa arsitektur alokasi memori dinamis jauh lebih efisien dan elastis dibandingkan array statis karena ruang memori hanya dibentuk secara presisi saat dibutuhkan dan langsung dikembalikan ke sistem saat tidak terpakai, di mana pengelolaannya menuntut ketelitian tinggi dalam menggunakan pointer bantuan penelusur data tanpa menggeser jangkar utama penunjuk awal agar rantai memori tidak terputus saat mengeksekusi operasi sisip terurut maupun hapus, yang mana struktur dasar ini juga menunjukkan keandalannya saat diterapkan untuk memecahkan masalah komputasi yang lebih kompleks seperti merepresentasikan persamaan bilangan polinomial secara ringkas dengan mengurutkan elemen otomatis dari pangkat tertinggi ke terendah, serta membuktikan kapasitasnya sebagai fondasi pembentukan struktur data Stack berkonsep Last In First Out (LIFO) melalui manipulasi Push dan Pop di awal list untuk menyelesaikan konversi bilangan desimal ke berbagai basis berkat kemampuannya membalikkan urutan keluaran secara otomatis, hingga pada akhirnya berhasil diekspansi menjadi purwarupa database yang sanggup mengelola rekaman majemuk entitas mahasiswa untuk menjaga keterurutan data berdasarkan nomor identitas sekaligus mengimplementasikan fitur pembaruan konten teks di dalam simpul secara langsung tanpa perlu merusak susunan tautan pointer pada keseluruhan sistem senarai berantai tersebut.